Guía de Estudio Completa — Redes Neuronales Recurrentes (RNN)

Curso: Inteligencia Artificial — ESPOL-FIEC — Prof. Enrique Peláez — PAO 2026-I

Elaborado a partir de: Anotaciones de clase (12, 13, 16), Guía Maestra, Preguntas de exámenes anteriores, Goodfellow *Deep Learning*, Hagan *NN Design*, Chollet *Deep Learning with Python*

---

0. Cómo usar esta guía

Este documento cubre las RNN con el mismo rigor matemático exigido en el curso: cada fórmula se deduce, se explica su intuición, y se resuelve con números reales — incluyendo el ejemplo oficial de clase (palabra **"amor"**) y varios problemas de examen ya resueltos paso a paso. Al final hay una sección de trampas conceptuales frecuentes y un formulario de repaso rápido.

Notación que se usará de forma consistente:

Símbolo	Significado
x_t	Vector de entrada en el instante (time-step) t
h_t	Estado oculto (memoria) en el instante t
y_t (o Y_t)	Salida de la red en el instante t
W_{xh}	Matriz de pesos entrada → capa oculta
W_{hh}	Matriz de pesos oculta(t-1) → oculta(t) (pesos **recurrentes**)
W_{hy}	Matriz de pesos oculta → salida
$b_h,\ b_y$	Sesgos (bias) de la capa oculta y de salida
f_1	Función de activación de la capa oculta (en el curso: **tanh** o **logística/sigmoide**, según el ejercicio)
f_2	Función de activación de la salida (en el curso: **softmax** para clasificación de vocabulario, **lineal** en ejercicios numéricos tipo "calcule $Y_{t=1}$")
T	Longitud de la secuencia

---

1. ¿Por qué existen las RNN? Motivación formal

Todo el curso construye modelos de la forma general

$$y = f(WX + b)$$

Lo único que cambia entre modelos es **la estructura de conexión** y **la forma del dato**:

Modelo	Forma del dato	Estructura de conexión
Regresión lineal/logística	escalar / vector fijo	densa, sin compartición
MLP	vector fijo	densa (todas contra todas)
CNN	volumen (imagen)	local, con **pesos compartidos espacialmente** (filtros)
RNN	**secuencia** (longitud variable)	recurrente, con **pesos compartidos en el tiempo**

El problema que resuelve la RNN: una MLP o una CNN reciben una entrada de tamaño fijo y no tienen memoria de entradas anteriores; cada ejemplo se procesa de forma independiente. Pero hay datos cuyo significado depende del **orden** y del **contexto histórico**: el precio del petróleo de mañana depende de la serie completa de precios pasados, la siguiente palabra de una oración depende de las palabras anteriores, el sentimiento de una reseña depende de la secuencia completa de tokens.

Idea central: en vez de tener una sola pasada hacia adelante, la RNN **reutiliza la misma capa oculta una vez por cada elemento de la secuencia**, y esa capa recibe, además de la entrada actual x_t, su propia salida del paso anterior h_{t-1}. Cada "copia" de la capa oculta a través del tiempo es formalmente **la misma red** (mismos pesos W_{xh}, W_{hh}, W_{hy}); lo único que cambia de una copia a otra es la entrada que recibe. Esto se llama **compartición de pesos en el tiempo** (*weight sharing*), exactamente análoga a la compartición espacial de un filtro convolucional en una CNN.

> **Frase de clase (profesor):** "el time-step 0 afecta para siempre la salida" — porque h_0 queda encapsulado dentro de h_1, que a su vez queda encapsulado dentro de h_2, y así sucesivamente. El estado oculto es una memoria comprimida y acumulativa de todo lo procesado hasta el instante actual.

---

2. Definición formal de la RNN (una capa)

2.1 Ecuaciones fundamentales

Para cada instante de tiempo $t = 0, 1, \ldots, T-1$:

$$\boxed{h_t = f_1\big(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\big)}$$

$$\boxed{y_t = f_2\big(W_{hy}\, h_t + b_y\big)}$$

con la condición inicial

$h_{-1} = \vec{0}$ (el estado oculto antes del primer time-step se inicializa en cero, salvo que se indique lo co

2.2 Notación con superíndices (versión oficial de la diapositiva de clase)

En la notación de Clase 16, si se escribe con sumatorias explícitas (por neurona i de la capa

oculta 1, recibiendo entradas X_j):

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(01)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

$$Y(t) = f_2\left(\sum_j w_{jk}^{(12)} h_j^{(1)}(t) + b_k^{(2)}\right)$$

Regla de lectura del superíndice: $w^{(mn)}$ significa "peso que va **de la capa m hacia la capa n** ". Así:

- $w^{(01)}$: de la **entrada** (capa 0) hacia la **capa oculta 1** — es W_{xh}.
- $w^{(11)}$: **recurrente**, de la capa oculta 1 en $t-1$ hacia sí misma en t — es W_{hh}.
- $w^{(12)}$: de la capa oculta 1 hacia la capa de **salida** (capa 2) — es W_{hy}.

Esta notación se vuelve indispensable cuando hay más de una capa recurrente (sección 3).

2.3 Funciones de activación usadas en el curso

Ubicación	Función típica en clase	Por qué
Capa(s) oculta(s)	**tanh** (rango $(-1, 1)$) — a veces **sigmoide/logística** en ejercicios numéricos de examen	Reescala la entrada evitando saturación extrema; centra la señal en 0
Capa de salida	**softmax** (si la salida es una distribución de probabilidad sobre un vocabulario) o **lineal** (si la salida es un valor numérico continuo, como en varios problemas de examen)	softmax porque $\sum_k P(y = k) = 1$; lineal cuando no hay restricción de probabilidad

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Trampa de examen #1 (muy frecuente): *usar softmax en la capa oculta*. Softmax **solo** se usa en la salida cuando ésta representa una distribución de probabilidad. La capa oculta usa tanh o sigmoide (o ReLU en arquitecturas modernas), nunca softmax.

2.4 Interpretación de cada componente

- x_t: la observación en el instante t (puede ser un vector one-hot de un carácter, un valor numérico como el precio del petróleo, un embedding de una palabra, etc.).
- h_t: la "memoria" — un resumen vectorial comprimido de $x_0, x_1, \ldots, x_t$. Por eso h_t depende recursivamente de h_{t-1}.
- W_{hh}: **es el corazón de la recurrencia.** Sin este término la red seria simplemente una MLP aplicada de forma independiente a cada x_t (sin memoria).
- b_h, b_y: cada neurona tiene su propio sesgo, igual que en MLP/CNN. **No se debe omitir en los cálculos** (error común señalado explícitamente por el profesor).

3. RNN de dos capas y RNN con *skipping*

3.1 Dos capas recurrentes

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(01)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

$$h_i^{(2)}(t) = f_2\left(\sum_j w_{ji}^{(12)} h_j^{(1)}(t) + \sum_j w_{ji}^{(22)} h_i^{(2)}(t-1) + b_i^{(2)}\right)$$

$$Y(t) = f_3\left(\sum_j w_{jk}^{(23)} h_j^{(2)}(t) + b_k^{(3)}\right)$$

Es decir: **cada capa oculta adicional tiene su propia recurrencia** (su propio término $h^{(\ell)}(t-1)$), y recibe como entrada la salida de la capa anterior en el **mismo** instante t (no la del instante anterior).

> **Advertencia del profesor:** rara vez se usan RNN con más de dos capas recurrentes apiladas, porque la complejidad computacional y el riesgo de overfitting/gradiente inestable "explotan". En la práctica moderna, para arquitecturas profundas se prefiere LSTM/GRU (sección 7) o Transformers antes que apilar muchas capas RNN simples.

3.2 RNN con *skipping* (conexiones de salto)

El *skipping* consiste en conectar una capa **directamente** con una capa no adyacente, para preservar información original que de otro modo se diluiría al pasar por varias transformaciones no lineales. Dos variantes vistas en clase:

- **Salto entrada → salida:** la salida incorpora directamente un término desde la entrada, además de lo que llega por la(s) capa(s) oculta(s):

$$Y(t) = f_3\left(\sum_j w_{jk}^{(23)} h_j^{(2)}(t) + \sum_j w_{ji}^{(13)} h_i^{(1)}(t) + b_k^{(3)}\right)$$

- **Salto entrada → segunda capa oculta:** la entrada original alimenta directamente la capa 2, sin pasar únicamente por la capa 1:

$$h_i^{(2)}(t) = f_2\left(\sum_j w_{ji}^{(12)} h_j^{(1)}(t) + \sum_j w_{ji}^{(02)} X_j(t) + \sum_j w_{ji}^{(22)} h_i^{(2)}(t-1) + b_i^{(2)}\right)$$

Idea conceptual: el *skipping* es el mismo principio que las conexiones residuales (*skip/residual connections*) en redes profundas: evita que la información se "diluya" o se pierda al atravesar demasiadas transformaciones no lineales.

4. Arquitecturas según entrada/salida (los "4 tipos de problema")

Tipo	Entrada	Salida	Ejemplo típico
One-to-one	un dato	una salida	Clasificación simple (equivalente a una MLP, no requiere recurrencia real)
One-to-many	un dato	una secuencia	Generar una descripción de texto a partir de una imagen
Many-to-one	una secuencia	una salida	Análisis de sentimiento de una oración completa
Many-to-many	una secuencia	una secuencia	Traducción automática, etiquetado de secuencias (POS tagging)

En **many-to-one**, típicamente solo se usa/evalúa y_T (la salida del último time-step), que ya contiene, comprimida en h_T, la información de toda la secuencia. En **many-to-many** se produce y se evalúa una salida y_t en cada instante (o, en variantes tipo *encoder-decoder*, primero se lee toda la secuencia de entrada y luego se genera la de salida).

---

5. Algoritmo de forward pass (pseudocódigo formal)

Entrada: secuencia $x_0, x_1, \ldots, x_{T-1}$; pesos W_{xh}, W_{hh}, W_{hy}; sesgos b_h, b_y.

```
h(-1) ← 0                                   # estado inicial
para t = 0 hasta T-1:
    z_h(t) ← W_xh · x(t) + W_hh · h(t-1) + b_h
    h(t)   ← f1( z_h(t) )                   # tanh o sigmoide
    z_y(t) ← W_hy · h(t) + b_y
    y(t)   ← f2( z_y(t) )                   # softmax o lineal
devolver { h(t), y(t) } para todo t
```

Puntos clave que se evalúan en examen:

1. Los pesos W_{xh}, W_{hh}, W_{hy} **son los mismos en cada iteración** — no cambian de un time-step a otro (compartición de pesos).
2. $h(-1) = 0$ es la convención estándar salvo indicación contraria.
3. El **orden de las operaciones importa**: primero se combina x_t y h_{t-1} linealmente, y **después** se aplica la no linealidad f_1. Nunca se aplica f_1 a x_t y a h_{t-1} por separado antes de sumarlos.

---

6. Función de costo para secuencias

Si en cada instante hay una salida deseada $\hat{y}_t$, la pérdida total de la secuencia es la suma (o el promedio) de las pérdidas por instante:

$$L = \sum_{t=0}^{T-1} L_t(y_t, \hat{y}_t)$$

- Si y_t es una distribución softmax sobre un vocabulario (predicción de siguiente carácter/palabra), se usa **entropía cruzada**:

$$L_t = -\sum_k \hat{y}_{t,k} \, \log(y_{t,k})$$

- Si y_t es un valor numérico continuo (regresión), se usa **error cuadrático medio**:

$$L_t = (y_t - \hat{y}_t)^2$$

---

7. Backpropagation Through Time (BPTT) — deducción formal

7.1 Idea: "desenrollar" la red en el tiempo

BPTT consiste en **desenrollar** (*unroll*) la RNN: se dibuja una copia de la celda recurrente por cada time-step, conectadas en cadena por h_t, y se trata el resultado como si fuera una red *feedforward* muy profunda (con tantas "capas" como time-steps), a la cual se le aplica backpropagation estándar (regla de la cadena), **con la restricción adicional de que los pesos son compartidos entre todas las copias**.

```
x0 → [celda] → h0 → [celda] → h1 → [celda] → h2 → ...
       ↓              ↓              ↓
       y0             y1             y2
```

Cada "[celda]" usa **el mismo** $W_{xh}, W_{hh}, W_{hy}, b_h, b_y$.

7.2 Gradiente respecto a los pesos de salida W_{hy}

Este es el caso más simple porque y_t depende de W_{hy} únicamente a través de h_t en ese mismo instante:

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=0}^{T-1} \frac{\partial L_t}{\partial y_t} \cdot \frac{\partial y_t}{\partial z_{y,t}} \cdot \frac{\partial z_{y,t}}{\partial W_{hy}} = \sum_{t=0}^{T-1} \delta_{y,t} \, h_t^\top$$

donde $\delta_{y,t} = \dfrac{\partial L_t}{\partial y_t} \cdot f_2'(z_{y,t})$ es el error retropropagado en la salida del instante t (idéntico al cálculo de "inicialización del gradiente" visto en MLP/CNN).

7.3 Gradiente respecto a los pesos recurrentes W_{hh} — el paso crítico

Aquí está la diferencia esencial respecto a una MLP: W_{hh} **se usa en todos los time-steps**, así que su gradiente total es la **suma de las contribuciones de cada instante**, y cada una de esas contribuciones se propaga **hacia atrás en el tiempo** a través de todos los h_k anteriores:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=0}^{T-1} \frac{\partial L_t}{\partial W_{hh}} = \sum_{t=0}^{T-1} \sum_{k=0}^{t} \frac{\partial L_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_k} \cdot \frac{\partial h_k}{\partial W_{hh}}$$

El término crítico es $\dfrac{\partial h_t}{\partial h_k}$, que por la regla de la cadena aplicada repetidamente es un **producto de Jacobianos**:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} \operatorname{diag}\left(f_1'(z_{h,i})\right) W_{hh}$$

Esta última expresión es **la fuente matemática exacta** del problema de gradiente desvaneciente/explosivo (sección 8).

7.4 Algoritmo BPTT (resumen operativo)

1. **Forward pass completo:** calcular y guardar h_t, y_t, $z_{h,t}$, $z_{y,t}$ para todo $t = 0, \dots, T-1$.
2. **Inicializar** el gradiente que fluye desde el futuro: $\dfrac{\partial L}{\partial h_T} = 0$ (no hay time-step después del último).
3. **Recorrer el tiempo hacia atrás**, desde $t = T-1$ hasta $t = 0$:
   - Calcular el error de salida $\delta_{y,t}$.
   - Sumar al gradiente total de h_t: la contribución de $\delta_{y,t}$ **más** la contribución que llega desde h_{t+1} (propagada por $W_{hh}^\top$).
   - Calcular $\delta_{h,t} = \left(\dfrac{\partial L}{\partial h_t}\right) \odot f_1'(z_{h,t})$.
   - Acumular gradientes: $\dfrac{\partial L}{\partial W_{xh}} \mathrel{+}= \delta_{h,t}\, x_t^\top$, $\quad \dfrac{\partial L}{\partial W_{hh}} \mathrel{+}= \delta_{h,t}\, h_{t-1}^\top$, $\quad \dfrac{\partial L}{\partial b_h} \mathrel{+}= \delta_{h,t}$.
4. Actualizar todos los pesos con los **gradientes acumulados** (suma sobre todos los time-steps), usando descenso de gradiente (o una variante como Adam):

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Trampa de examen #2: creer que se actualiza un W_{hh} distinto por cada time-step. **Falso**: hay un único W_{hh} (y un único W_{xh}, W_{hy}); lo que se acumula por time-step son los **gradientes**, que luego se suman antes de una única actualización.

---

8. El problema del gradiente desvaneciente / explosivo (deducción formal)

De la sección 7.3:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \operatorname{diag}\left(f_1'(z_{h,i})\right) W_{hh}$$

Este es un producto de $(t-k)$ matrices. Intuitivamente, para una versión escalar simplificada (una sola neurona oculta), esto se reduce a:

$$\frac{\partial h_t}{\partial h_k} \approx \left(f_1' \cdot W_{hh}\right)^{t-k}$$

Análisis de casos:

- Si $|f_1' \cdot W_{hh}| < 1$: el producto tiende a **0** exponencialmente rápido conforme $t - k$ crece → **gradiente desvaneciente (vanishing gradient)**. La red "olvida" dependencias lejanas: los primeros time-steps dejan de influir en el error de los últimos, y sus pesos casi no se actualizan.
- Si $|f_1' \cdot W_{hh}| > 1$: el producto tiende a **infinito** → **gradiente explosivo (exploding gradient)**. Las actualizaciones se vuelven numéricamente inestables (valores NaN, oscilaciones).

Por qué tanh/sigmoide agravan el problema: $f_1' = \tanh'(z) = 1 - \tanh^2(z) \in (0, 1]$, y $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \in (0, 0.25]$. Como estas derivadas son **siempre menores o iguales a 1** (y la de sigmoide, menores o iguales a 0.25), multiplicarlas repetidamente casi garantiza que el producto tienda a 0 salvo que W_{hh} compense con valores grandes (lo cual, a su vez, arriesga explosión). Este es exactamente el mecanismo que describe Goodfellow: multiplicaciones repetidas por W^t hacen que los componentes correspondientes al mayor valor propio (eigenvalor) dominen y el resto se desvanezca.

Trampa de examen #3 (muy citada en preguntas anteriores): atribuir el desvanecimiento del gradiente a que la activación de salida es *softmax*. **Esto es incorrecto.** El problema se origina en la **multiplicación repetida por W_{hh} y por las derivadas de la activación oculta** (tanh/sigmoide) a lo largo del tiempo — **no** en la función softmax de salida, que solo se aplica una vez por time-step y no participa en la cadena recurrente de derivadas $\partial h_t / \partial h_k$.

Consecuencia práctica: las RNN simples (*vanilla RNN*) tienen dificultad para aprender **dependencias de largo plazo** (relacionar la primera palabra de un párrafo con la última). De ahí que, según las anotaciones de clase, "las RNN solían perder la relación entre la primera y la última palabra (de ahí que 'alucinen')".

Mitigaciones estándar:

Técnica	Ataca	Cómo
Gradient clipping	Explosión	Recortar la norma del gradiente a un máximo antes de actualizar
Inicialización cuidadosa de W_{hh} (p. ej. ortogonal)	Ambos	Mantener el valor propio dominante cercano a 1
LSTM / GRU	Desvanecimiento (principalmente)	Introducen "compuertas" y una ruta aditiva (no puramente multiplicativa) para el flujo del gradiente
Arquitecturas más recientes (Transformers)	Ambos, y elimina la recurrencia secuencial	Atención en vez de recurrencia

---

9. LSTM (Long Short-Term Memory)

9.1 Motivación formal

El problema de la RNN simple es que h_t se recalcula por completo en cada paso mediante una transformación **no lineal multiplicativa** (W_{hh} seguido de tanh), lo que fuerza al gradiente a atravesar $(t - k)$ multiplicaciones. LSTM introduce una segunda variable de estado, la **celda de memoria** c_t, que se actualiza de forma mayormente **aditiva** (no solo

multiplicativa), creando una "autopista" por la que el gradiente puede fluir sin desvanecerse tan rápido.

9.2 Ecuaciones completas

Sea $[h_{t-1}, x_t]$ la concatenación del estado oculto anterior y la entrada actual. LSTM define **tres compuertas** (gates) y una **celda candidata**:

$$\text{Compuerta de olvido (forget gate):} \quad f_t = \sigma\big(W_f\,[h_{t-1}, x_t] + b_f\big)$$

$$\text{Compuerta de entrada (input gate):} \quad i_t = \sigma\big(W_i\,[h_{t-1}, x_t] + b_i\big)$$

$$\text{Compuerta de salida (output gate):} \quad o_t = \sigma\big(W_o\,[h_{t-1}, x_t] + b_o\big)$$

$$\text{Celda candidata:} \quad \tilde{c}_t = \tanh\big(W_c\,[h_{t-1}, x_t] + b_c\big)$$

Actualización de la celda de memoria (clave del diseño):

$$\boxed{c_t = f_t \odot c_{t-1} \;+\; i_t \odot \tilde{c}_t}$$

Estado oculto de salida:

$$\boxed{h_t = o_t \odot \tanh(c_t)}$$

($\odot$ = producto elemento a elemento / Hadamard)

9.3 Interpretación de cada compuerta

Compuerta	Rango	Pregunta que responde
f_t (olvido)	$(0, 1)$	¿Cuánto de la memoria anterior c_{t-1} debo **conservar**? ($f_t \approx 1$: conservar todo; $f_t \approx 0$: olvidar todo)
i_t (entrada)	$(0, 1)$	¿Cuánta información **nueva** ($\tilde{c}_t$) debo incorporar a la memoria?
o_t (salida)	$(0, 1)$	¿Cuánto de la memoria interna c_t debo **exponer** como estado oculto h_t (visible a la siguiente capa/salida)?

9.4 Por qué mitiga el desvanecimiento del gradiente

Al derivar $\partial c_t / \partial c_{t-1}$ de la ecuación $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t + (\text{términos adicionales por la dependencia de } f_t, i_t, \tilde{c}_t \text{ en } c_{t-1} \text{ vía } h_{t-1})$$

El término dominante es simplemente f_t (la compuerta de olvido), **no un producto de W_{hh} repetido**. Si la red aprende $f_t \approx 1$ para una dependencia importante, el gradiente **fluye casi sin atenuarse** a través de muchos time-steps mediante esta ruta aditiva. Esto es lo que permite a LSTM capturar dependencias de largo plazo mucho mejor que una RNN simple.

Trampa de examen #4: decir que LSTM **elimina por completo** el problema del gradiente desvaneciente/explosivo. **Es incorrecto** — LSTM lo **mitiga considerablemente**, pero en secuencias extremadamente largas o con muchas capas apiladas el problema puede seguir presente en menor medida (de ahí que, según la pregunta de examen 108, LSTM/GRU son efectivas "porque tienen mecanismos para recordar información a largo plazo", no porque lo resuelvan de forma absoluta).

---

10. GRU (Gated Recurrent Unit)

Versión simplificada de LSTM: **fusiona** la celda de memoria y el estado oculto en uno solo (h_t), y usa **dos** compuertas en vez de tres.

$$\text{Compuerta de actualización (update gate):} \quad z_t = \sigma\big(W_z\,[h_{t-1}, x_t]\big)$$

$$\text{Compuerta de reinicio (reset gate):} \quad r_t = \sigma\big(W_r\,[h_{t-1}, x_t]\big)$$

$$\text{Estado candidato:} \quad \tilde{h}_t = \tanh\big(W\,[r_t \odot h_{t-1},\ x_t]\big)$$

$$\boxed{h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t}$$

Interpretación: z_t controla la mezcla entre "conservar el estado anterior" y "reemplazarlo por el nuevo candidato"; r_t controla cuánto del pasado se usa para calcular ese candidato. GRU tiene **menos parámetros** que LSTM (no tiene celda de memoria separada ni compuerta de salida), por lo que suele entrenar más rápido con desempeño comparable en muchas tareas.

10.1 Tabla comparativa RNN vs LSTM vs GRU

Característica	RNN simple	LSTM	GRU
Estados internos	h_t	h_t y c_t	solo h_t
Compuertas	ninguna	3 (olvido, entrada, salida)	2 (actualización, reinicio)
Parámetros	menos	más	intermedio
Dependencias largas	débil (gradiente se desvanece rápido)	fuerte	fuerte
Ruta del gradiente	puramente multiplicativa	mayormente aditiva (vía c_t)	mayormente aditiva (vía $(1 - z_t)$)

---

11. Ejemplo numérico oficial de clase: predicción de carácter con la palabra "amor"

Este es el ejercicio de referencia explícito de la Clase 16 (diapositivas), el tipo de cálculo más

probable en examen.

Planteamiento: vocabulario (corpus) $= \{a, m, o, r\}$, es decir 4 símbolos posibles. Se ingresa la secuencia "a, m, o" y se desea que la red prediga que el siguiente carácter es "r". La capa oculta usa **tanh**; la capa de salida usa **softmax** (porque la salida es una distribución de probabilidad sobre las 4 letras del vocabulario).

Ecuaciones aplicadas:

$$h_t = \tanh\left(W_{xh}\, X_t + W_{hh}\, h_{t-1} + b_h\right), \qquad Y_t = \text{softmax}\left(W_{hy}\, h_t + b_y\right)$$

Paso $t = 0$ (entrada = "a", con $h_{-1} = \vec{0}$; los pesos son inicializados aleatoriamente y los valores pre-activación de la diapositiva son):

$$h_0 = \tanh\Big(\,[0.287027,\ 0.902874,\ 0.537524]^\top + [0.567,\ 0.567,\ 0.567]^\top\,\Big)$$

$$h_0 = \tanh\left([0.854027,\ 1.469874,\ 1.104524]^\top\right) = [0.69316804,\ 0.89955366,\ 0.80211840]^\top$$

Paso $t = 1$ (entrada = "m"; ahora se combina $W_{hh}h_0$ con la contribución de la nueva entrada $W_{xh}X_1$):

$$h_1 = \tanh\Big(\,[0.863013,\ 0.951149,\ 0.90954]^\top + [0.846060,\ 0.871522,\ 0.092240]^\top\,\Big)$$

$$h_1 = \tanh\left([1.709073,\ 1.822671,\ 1.001780]^\top\right) = [0.93653372,\ 0.94910403,\ 0.76234057]^\top$$

Salida final (tras procesar también "o" para llegar al h final, se calcula el vector de logits $z_y = W_{hy}h + b_y$ y se aplica softmax; el vector de logits de la diapositiva es):

$$z_y = [1.90607732,\ 1.13779113,\ 0.95666016,\ 1.27422602]^\top$$

$$Y = \text{softmax}(z_y) = [0.419748,\ 0.194682,\ 0.162429,\ 0.223141]^\top$$

Interpretación del resultado: la componente más alta del vector softmax es 0.419748 (primera posición). Como el vector softmax asigna una probabilidad a **cada letra del vocabulario en el orden** $\{a, m, o, r\}$, esto significa que, con estos pesos (aleatorios, sin entrenar todavía), la red predice "a" como carácter más probable — lo cual **no** es la letra correcta ("r"). Esto ilustra por qué el entrenamiento (ajuste de pesos vía BPTT minimizando la entropía cruzada entre Y y el one-hot de "r") es necesario: con los pesos iniciales aleatorios la predicción es esencialmente arbitraria, y el aprendizaje debe desplazar la masa de probabilidad hacia la componente correcta.

Verificación de cómo se leen las sumas dentro del tanh: el primer vector de cada suma corresponde a $W_{hh}\ h_{t-1}$ (contribución recurrente/memoria) y el segundo a $W_{xh}\ X_t + b_h$ (contribución de la entrada nueva más el sesgo) — o viceversa, según cómo se hayan agrupado los términos en la diapositiva; lo importante conceptualmente es que **ambas contribuciones se suman antes de aplicar tanh**, nunca se aplica tanh por separado a cada una.

---

12. Problemas de examen resueltos paso a paso (formato "calcule Y en $t = 1$")

Este formato aparece repetidamente en exámenes anteriores. La estructura es siempre la misma: se da una RNN de **una sola neurona oculta** (versión escalar), con activación **logística (sigmoide)** en la capa oculta y activación **lineal** en la salida:

$$h_t = \sigma\big(W_{xh} X_t + W_{hh} h_{t-1} + h_{bias}\big), \qquad Y_t = W_{hy}\, h_t + Y_{bias}$$

con $h_{-1} = 0$.

Problema resuelto A

Datos: $W_{xh} = 0.8,\ W_{hh} = 1.5,\ W_{hy} = -0.5,\ h_{bias} = 1.2,\ Y_{bias} = 0.8$. Entradas $X_0 = 6,\ X_1 = 2,\ X_2 = -1$. Calcular Y en $t = 1$.

Paso $t = 0$:

$$h_0 = \sigma(0.8 \times 6 + 1.5 \times 0 + 1.2) = \sigma(4.8 + 1.2) = \sigma(6.0) = \frac{1}{1 + e^{-6.0}} \approx 0.997527$$

Paso $t = 1$:

$$h_1 = \sigma(0.8 \times 2 + 1.5 \times 0.997527 + 1.2) = \sigma(1.6 + 1.496291 + 1.2) = \sigma(4.296291)$$

$$h_1 = \frac{1}{1 + e^{-4.296291}} \approx 0.986566$$

Salida:

$$Y_1 = W_{hy}\, h_1 + Y_{bias} = -0.5 \times 0.986566 + 0.8 = -0.493283 + 0.8 \approx \boxed{0.3067 \approx 0.31}$$

Problema resuelto B

Datos: $W_{xh} = 0.5,\ W_{hh} = -1.0,\ W_{hy} = -0.7,\ h_{bias} = -1.0,\ Y_{bias} = 0.0$. Entradas $X_0 = 9,\ X_1 = 4,\ X_2 = -2$. Calcular Y en $t = 1$.

Paso $t = 0$:

$$h_0 = \sigma(0.5 \times 9 + (-1.0) \times 0 + (-1.0)) = \sigma(4.5 - 1.0) = \sigma(3.5) \approx 0.970688$$

Paso $t = 1$:

$$h_1 = \sigma(0.5 \times 4 + (-1.0) \times 0.970688 + (-1.0)) = \sigma(2.0 - 0.970688 - 1.0) = \sigma(0.029312) \approx 0.50732$$

Salida:

$$Y_1 = -0.7 \times 0.507327 + 0.0 \approx \boxed{-0.3551 \approx -0.36}$$

Problema resuelto C

Datos: $W_{xh} = 1.2,\ W_{hh} = -2.0,\ W_{hy} = 1.0,\ h_{bias} = 0.8,\ Y_{bias} = 0.0$. Entradas $X_0 = 7,\ X_1 = 3,\ X_2 = -4$. Calcular Y en $t = 1$.

Paso $t = 0$:

$$h_0 = \sigma(1.2 \times 7 + (-2.0) \times 0 + 0.8) = \sigma(8.4 + 0.8) = \sigma(9.2) \approx 0.999899$$

Paso $t = 1$:

$$h_1 = \sigma(1.2 \times 3 + (-2.0) \times 0.999899 + 0.8) = \sigma(3.6 - 1.999798 + 0.8) = \sigma(2.400202) \approx 0.916827$$

Salida:

$$Y_1 = 1.0 \times 0.916827 + 0.0 \approx \boxed{0.9168 \approx 0.92}$$

Método general para este tipo de ejercicio (algoritmo de resolución)

1. Identificar los 5 parámetros: $W_{xh}, W_{hh}, W_{hy}, h_{bias}, Y_{bias}$.
2. Fijar $h_{-1} = 0$ salvo indicación contraria.
3. Calcular h_0 con la fórmula $\sigma(W_{xh}X_0 + W_{hh}h_{-1} + h_{bias})$.
4. Calcular h_1 **usando h_0 ya calculado** (nunca X_0 de nuevo): $\sigma(W_{xh}X_1 + W_{hh}h_0 + h_{bias})$.
5. Si se pide Y_t para $t > 1$, seguir encadenando $h_2, h_3, \ldots$ de la misma manera antes de aplicar la salida lineal.
6. Aplicar la salida: $Y_t = W_{hy}h_t + Y_{bias}$ (lineal, **sin** sigmoide/softmax en este tipo de ejercicio particular).

Error común a evitar: olvidar que h_1 se calcula con h_0 (no con h_{-1}) — es decir, no "reiniciar" la recurrencia en cada paso.

---

13. Cuestionario conceptual formal (con justificación) — basado en preguntas de exámenes anteriores

P1. ¿Es verdad que la propagación del error hacia atrás en el tiempo, usando la regla de la cadena, permite determinar cuánto contribuyó cada neurona al error total? Verdadero. Esto es exactamente BPTT (sección 7): se retropropaga $\delta_{h,t}$ a través de cada instante, acumulando la contribución de cada parámetro compartido.

P2. ¿El desvanecimiento del gradiente en RNN se debe a que la activación es softmax? Falso. Se debe a la multiplicación repetida de W_{hh} y de las derivadas de la activación **oculta** (tanh/sigmoide) a lo largo de $(t - k)$ pasos (sección 8). Softmax es la activación de **salida**, y no interviene en la cadena recurrente $\partial h_t / \partial h_k$.

P3. ¿LSTM resuelve por completo el problema del gradiente desvaneciente? Falso (lo **mitiga**, no lo elimina). Ver sección 9.4.

P4. ¿Una medida del número de capas ocultas necesarias en una RNN es la "longitud del contexto"? Esta afirmación **confunde dos conceptos distintos**: la *longitud del contexto* (cuántos time-steps hacia atrás la red logra "recordar" efectivamente) es una propiedad

temporal, relacionada con qué tan bien fluye el gradiente a través del tiempo (sección 8) — **no** es lo mismo que el número de capas ocultas apiladas (profundidad espacial de la red en un instante dado). Es una trampa conceptual frecuente en el examen: profundidad temporal (time-steps) ≠ profundidad de capas (arquitectura).

P5. En las RNN, si al aplicar la regla de la cadena cualquiera de los gradientes intermedios se acerca a 0, ¿todos los gradientes siguientes también tenderán a 0 exponencialmente rápido por la multiplicación? Verdadero, es consecuencia directa de que $\partial h_t / \partial h_k$ es un **producto** de $(t-k)$ factores (sección 7.3): si uno de los factores es muy pequeño, el producto completo se ve dominado por ese factor y decae rápidamente; con más factores pequeños adicionales, el decaimiento se vuelve exponencial en el número de pasos.

P6. ¿La arquitectura de la RNN necesita explícitamente que las entradas sean secuencias de longitud fija? Falso. Una de las ventajas centrales de la RNN frente a la MLP es que puede procesar secuencias de **longitud variable**, porque la misma celda (mismos pesos) se reutiliza tantas veces como elementos tenga la secuencia.

P7. ¿Cómo se maneja la dependencia de largo plazo en las RNN? Mediante arquitecturas especializadas como **LSTM o GRU**, que introducen compuertas y una ruta de flujo de gradiente mayormente aditiva (secciones 9 y 10). Incrementar el número de neuronas ocultas o cambiar la tasa de aprendizaje **no** resuelve el problema estructural del desvanecimiento del gradiente.

P8. ¿Qué caracteriza a los datos secuenciales frente a otros tipos de datos? La **dependencia temporal** entre las observaciones: el orden y la relación entre elementos consecutivos contienen información relevante que se pierde si se trata cada dato como independiente (a diferencia de, por ejemplo, filas independientes en un dataset tabular).

P9. ¿RNN puede modelar una secuencia mientras que una MLP no?Sí, porque la MLP no tiene mecanismo de memoria: procesa cada entrada de forma aislada con pesos fijos, sin ningún término que dependa de entradas anteriores. La RNN introduce $W_{hh}h_{t-1}$, que es precisamente el mecanismo de memoria/recurrencia ausente en la MLP.

---

14. Relación RNN → NLP (contexto necesario para entender el ejemplo "amor")

Para que una RNN procese texto, **las palabras/caracteres deben convertirse en números** antes de entrar como x_t:

- **Tokenización:** dividir el texto en unidades (caracteres, palabras, subpalabras). Ejemplo: "amo la IA" → ["amo", "la", "IA"].
- **One-hot:** cada símbolo del vocabulario se representa como un vector con un 1 en su posición y 0 en el resto (es la representación usada en el ejemplo "amor": vocabulario $\{a, m, o, r\}$ → vectores de dimensión 4).
- **N-gramas:** secuencias de n tokens consecutivos; a mayor n, mayor problema de **sparsity** (escasez de datos, pues combinaciones largas rara vez se repiten en el corpus).
- **TF-IDF:** $TF(t,d) = \dfrac{\#\text{apariciones de } t \text{ en } d}{\#\text{términos en } d}$, $\quad IDF(t) = \log\left(\dfrac{N}{df(t)}\right)$, $TFIDF = TF \cdot IDF$. Pondera un término por su frecuencia local, penalizando los que aparecen en casi todos los documentos (poco informativos).
- **Embeddings:** representaciones vectoriales densas (no one-hot) que capturan relaciones semánticas; palabras similares quedan cerca en el espacio vectorial.

Estáticos (Word2Vec, GloVe) vs. contextuales (ELMo, BERT — el mismo token puede tener vectores distintos según el contexto).

Por qué esto importa para RNN: en el ejemplo "amor", cada letra ingresa como vector one-hot de dimensión 4 (tamaño del vocabulario), y W_{xh} transforma ese vector disperso en la dimensión del estado oculto.

---

15. Formulario de repaso rápido (una página)

Forward (una capa):

$$h_t = f_1(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad y_t = f_2(W_{hy}h_t + b_y), \qquad h_{-1} = \vec{0}$$

Dos capas:

$$h_t^{(1)} = f_1\big(W_{xh}x_t + W^{(11)}h_{t-1}^{(1)} + b^{(1)}\big), \quad h_t^{(2)} = f_2\big(W^{(12)}h_t^{(1)} + W^{(22)}h_{t-1}^{(2)} + b^{(2)}\big), \quad y_t = f_3\big(W^{(23)}h_t^{(2)} +$$

BPTT — Jacobiano recurrente (fuente del vanishing/exploding):

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \operatorname{diag}(f_1'(z_{h,i}))\, W_{hh}$$

LSTM:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),\ i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),\ o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$$

GRU:

$$z_t = \sigma(W_z[h_{t-1}, x_t]),\ r_t = \sigma(W_r[h_{t-1}, x_t]),\ \tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Activaciones:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \operatorname{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Checklist anti-trampas para el examen:

- [ ] Nunca omitir h_{t-1} ni el bias en las ecuaciones.
- [ ] La capa oculta usa tanh/sigmoide; softmax va **solo** en la salida (si es clasificación).
- [ ] Los pesos W_{xh}, W_{hh}, W_{hy} son **compartidos** en todos los time-steps (no hay uno distinto por paso).

- [ ] El desvanecimiento del gradiente se debe a W_{hh} y f_1' multiplicados repetidamente — **no** a softmax.
- [ ] LSTM/GRU **mitigan**, no eliminan, el problema del gradiente.
- [ ] h_1 se calcula usando h_0, no reiniciando desde h_{-1}.
- [ ] Longitud del contexto (temporal) ≠ número de capas ocultas (arquitectura).
- [ ] En RNN de 2 capas, cada capa tiene su **propia** recurrencia (su propio $h^{(\ell)}(t-1)$).

---

16. Ejercicios propuestos para autoevaluación (sin resolver — practique antes de ver la solución)

1. Con $W_{xh}=1.0,\ W_{hh}=0.5,\ W_{hy}=2.0,\ h_{bias}=0.0,\ Y_{bias}=-1.0$ y entradas $X_0=1,\ X_1=-2,\ X_2=3$ (activación oculta logística, salida lineal), calcule Y_1 y Y_2.
2. Explique, usando la ecuación de $\partial h_t/\partial h_k$, por qué inicializar W_{hh} con valores propios cercanos a 1 ayuda a mitigar (aunque no elimina) el desvanecimiento del gradiente.
3. Dibuje la celda LSTM completa etiquetando las 3 compuertas, la celda candidata, $c_{t-1}\to c_t$ y $h_{t-1}\to h_t$, e indique la ecuación asociada a cada flecha.
4. Compare, en una tabla, el número de matrices de pesos entrenables en una RNN simple de una capa vs. una LSTM de una capa (mismo tamaño de estado oculto), y explique la diferencia de costo computacional.
5. Para el ejemplo "amor": si en vez de softmax se usara sigmoide independiente por componente en la salida, ¿seguiría siendo una distribución de probabilidad válida sobre el vocabulario? Justifique formalmente.

Fin de la guía. Documento preparado integrando: Anotaciones de clases 12/13/16, Guía Maestra del curso, banco de preguntas de exámenes anteriores, Goodfellow — Deep Learning (cap. de RNN y gradientes), y Chollet — Deep Learning with Python (RNN/LSTM/GRU aplicadas a NLP).